

TECHNICAL REPORT · 2026 FIELD GUIDE

Diffusion LLM 入门、 技术栈与产业前景

从 0 到 1 建立扩散语言模型的完整心智模型：数学直觉、核心架构、训练与推理、工程部署、国内外厂商版图、发展方向与风险边界。

Kami Research Notes

V1.0 · 2026-05-22

来源核验截至 2026-05-22，面向技术读者与研究型工程团队

目录

01	执行摘要：为什么 diffusion LLM 值得重新理解语言生成	03
02	从自回归到扩散：两种生成范式的根本差别	04
03	数学骨架：前向加噪、反向去噪与掩码扩散	06
04	完整架构：模型、采样器、缓存、控制器与部署链路	08
05	训练技术栈：预训练、后训练、RL、评测与数据工程	11
06	推理技术栈：并行解码、重掩码、KV Cache 与质量控制	14
07	主流模型谱系：从 D3PM 到 LLaDA、Dream、Mercury、Gemini Diffusion	17
08	国内外厂商与开源生态：谁在做什么	21
09	能力边界与失败模式：速度不是全部答案	24
10	未来路线图：2026-2028 的五条主线	26
11	学习路径、术语表与参考资料	28

执行摘要

diffusion LLM 指把语言生成写成「从噪声或掩码状态逐步恢复文本」的模型族。它不是把 Transformer 换掉，而是把训练目标、注意力结构、采样顺序和服务系统重新组织：模型可以在一个候选序列上同时观察前后文，反复修正不确定位置，并用更少的串行步数生成多 token 输出。

五个结论先行

- 主流路线已经从连续嵌入扩散转向离散掩码扩散。D3PM 证明离散扩散可处理文本，Diffusion-LM 展示可控生成，SEDD 与 MDLM 改善训练目标，LLaDA 与 Dream 把规模推到 7B-8B 级别。[S01, S02, S03, S04, S05, S11]
- 速度优势来自「少串行步数 + 多位置并行修正」，不是免费午餐。开源模型若直接全序列多轮去噪，可能因重复全前向、缺少传统 KV Cache 而慢；Fast-dLLM、Block Diffusion、DPad 等工作都在补这个工程缺口。[S17-S19]
- 商业化最强信号来自 Inception 和 Google。Inception 的 Mercury 系列已经走向 API 与企业场景；Google DeepMind 的 Gemini Diffusion 是实验研究模型，官方展示了 1479 tokens/s 的采样速度和接近 Gemini 2.0 Flash-Lite 的多项代码 benchmark 表现。[S12-S14]
- 国内公开工作集中在 Ant/RUC、HKU/Huawei、Tencent AI Lab 等线索。LLaDA、LLaDA-MoE、LLaDA2.0、Dream、DiffuLLaMA 等工作构成国内外合作与开源扩散语言模型的主体。[S06, S08, S09, S10, S11, S15]
- 2026 年的关键问题不再是「能不能生成流畅文本」，而是「能否稳定服务真实工作负载」。系统会围绕可变长度、严格格式、代码与编辑、长上下文、RL 对齐、缓存复用、模型路由与多模态统一继续演进。

本文档回答的问题

READER CONTRACT

读完后你应能回答四件事：一是 diffusion LLM 与 GPT 式自回归模型到底差在哪；二是从数据、训练、采样到部署需要哪些组件；三是主流论文、开源模型和厂商产品各自解决了什么问题；四是判断一个 diffusion LLM 项目是否值得投入时该看哪些指标。

适用读者

如果你是刚接触这个方向的工程师，可以把前四章当作入门教材；如果你已熟悉 LLM 训练和推理，可以重点读第五到第十章；如果你负责技术路线判断，可以直接看厂商版图、风险边界与路线图。本文

把推测和事实分开：模型参数、发布日期、性能数字只在可核验来源中出现，未公开的公司内部项目不写成确定事实。

02 · PARADIGM

| 从自回归到扩散

自回归模型像一个严格按时间推进的写作者：已经写出的 token 固定下来，下一步只预测一个或少数几个新 token。扩散语言模型像一个反复修订的编辑器：先给出一个带有大量空位或噪声的草稿，再用多轮去噪把不确定位置逐步填实。

同一个 Transformer，三种概率分解

范式	训练目标	生成顺序	优势	短板
自回归 LM	预测下一个 token： $p(x_i x_{<i})$	左到右，一步依赖上一步	似然清晰、KV Cache 成熟、生态完整	长输出串行，回头改写困难，填中间需要重新设计提示
掩码 LM	预测被随机遮住的 token	通常一次性补全，不作为生成主范式	双向上下文，理解任务强	单步填空难以形成高质量长文本
扩散 LM	学习从噪声或 [MASK] 逐步恢复数据	任意顺序或置信度驱动，多位置并行	可编辑、可填中间、可做质量速度权衡	采样器、缓存、长度控制、对齐训练仍在快速变化

为什么语言扩散比图像扩散难

图像像素在连续空间中可以加高斯噪声，去噪网络每一步预测一个连续值。语言 token 是离散符号，不能自然地把「猫」和「经济学」平均成一个半猫半经济学的 token。因此早期路线有两类：一类把 token 映射到连续嵌入空间，在向量上做扩散，再把向量舍入回词表；另一类直接在词表上定义离散转移，例如把 token 逐步替换成 [MASK] 或按转移矩阵变换。后者更接近当前大规模 diffusion LLM 的主流。

Diffusion-LM 选择连续嵌入路线，优势是可借鉴视觉扩散的数学工具，并能通过梯度引导做细粒度控制；问题是最后的 rounding 步骤会把连续向量映射回离散 token，容易引入误差。[S02] D3PM、SEDD、MDLM、LLaDA、Dream 等工作把扩散过程放回离散空间，降低了符号语言的错配。[S01, S03, S04, S05, S11]

FIGURE 1 AR VS MASKED DIFFUSION



图 1: 扩散语言模型把生成顺序从「严格左到右」变成「由噪声水平和置信度决定」。

直觉公式

自回归模型直接建模 $p(x) = \prod p(x_i | x_{<i})$ 。掩码扩散模型引入时间 t , 把原始序列 x_0 逐步腐蚀成 x_t , 再训练模型估计 $p_\theta(x_0 | x_t, t)$ 或反向转移 $p_\theta(x_s | x_t)$ 。如果噪声是 [MASK], 训练时随机掩掉一部分 token; 推理时从全 [MASK] 或部分 [MASK] 开始, 多轮预测并释放 token。

判断 diffusion LLM 的第一原则: 不要只问「是否并行」。应问「并行到什么粒度、需要多少去噪声、每步是否复用缓存、质量下降由谁控制、输出长度如何决定」。

03 · MATHEMATICAL SKELETON

数学骨架

入门 diffusion LLM 不需要先推完整 ELBO, 但必须知道三个对象: 前向过程 q 、反向模型 p_θ 、采样策略。多数工程难题都能回到这三个对象。

前向过程: 把文本变成可恢复的损坏样本

前向过程 $q(x_t | x_0)$ 是人为设计的损坏规则。连续扩散常用高斯噪声; 离散扩散需要转移矩阵或吸收态。D3PM 提出了结构化离散转移矩阵, 包括基于邻近 token 的转移和吸收态 [MASK], 并指出吸收态可以连接到 BERT 式掩码语言建模与自回归生成。[S01]

噪声机制	怎么损坏文本	适合场景	代表工作
连续嵌入噪声	把 token embedding 加高斯噪声，再学习去噪向量	可控生成、梯度引导、早期探索	Diffusion-LM
均匀离散替换	按词表概率把 token 替换成其他 token	理论分析、离散 score 估计	D3PM、SEDD
吸收态掩码	逐步把 token 替换成 [MASK]	大规模语言模型、任意顺序生成、填中间	MDLM、LLaDA、Dream
块级扩散	块之间自回归，块内部扩散去噪	可变长度、缓存复用、工程部署	Block Diffusion

反向过程：学习从损坏样本恢复原文

训练时，模型看到 x_t 、时间 t 和条件输入 c ，输出每个位置属于词表中各 token 的概率。若采用 x_0 -prediction，模型直接预测原始 token；若采用 score 或 ratio 估计，模型学习「把当前离散状态改成另一个状态的相对倾向」。SEDD 的贡献在于把 score matching 推广到离散空间，用 score entropy loss 改善离散语言扩散的训练效果。[S03]

MDLM 的重要性在于简化了掩码离散扩散的目标：作者把目标写成一组经典 masked language modeling loss 的混合，并用现代训练 recipe 证明掩码扩散的 perplexity 可以接近自回归模型。[S04] 对学习者来说，这个结果改变了路线判断：语言扩散不是必须先发明全新 Transformer，而是可以用熟悉的 token 预测、掩码比例、噪声 schedule 和采样器形成有效系统。

采样策略：决定「先填哪里，何时停止」

推理时最简单的采样是从全 [MASK] 序列开始，重复执行以下步骤：模型预测每个位置的 token 分布；根据置信度选出一批位置固定；其余位置继续保持或重新变成 [MASK]。Dream 的实现暴露了多种 remasking 策略，包括随机顺序、top1 置信度、top1 与 top2 margin、熵值等；参数 steps 直接控制去噪轮数，轮数少更快，轮数多质量更稳。[S11]

给定 prompt 和目标生成长度 L:

1. 初始化 $y = [\text{MASK}] \times L$

2. 对 step = T ... 1:

a. logits = denoiser(prompt, y, step)

b. 计算每个位置的置信度 confidence

c. 固定高置信位置，低置信位置继续 mask

d. 可选：温度、top-p、格式约束、奖励引导

3. 拼接 prompt 与 y，得到最终输出

关键判断

扩散模型不是「一次性生成整段文本」。它是用少于输出长度的去噪轮数生成整段文本。若 T 仍然很大、每轮都跑完整 Transformer、缓存复用差，理论并行性可能在真实延迟中被抵消。

完整架构

diffusion LLM 的系统架构至少包含五层: 数据与 tokenizer、去噪 Transformer、噪声与损失、采样器、服务层。只改模型文件而不改采样与服务, 通常得不到生产级收益。

FIGURE 2 DIFFUSION LLM STACK

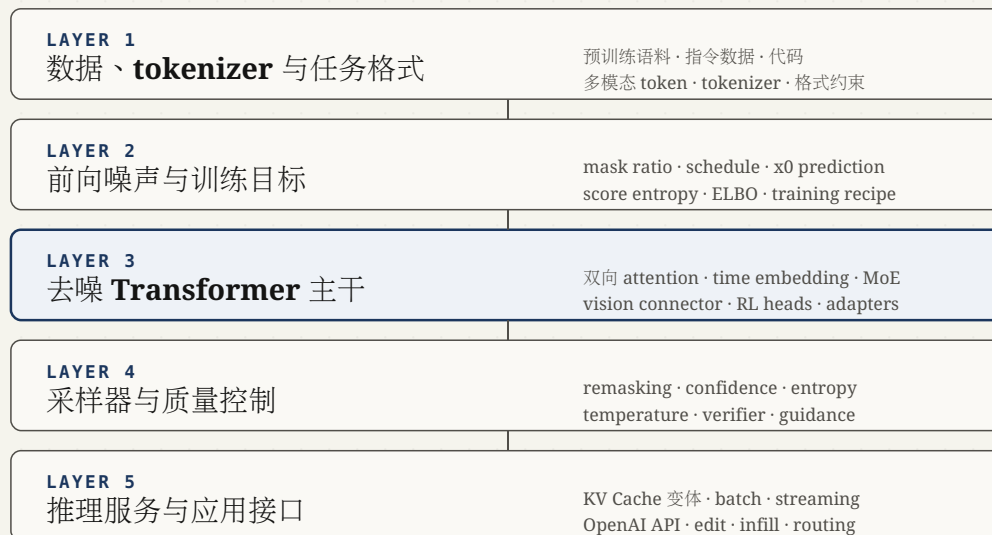


图 2: diffusion LLM 的完整栈。去噪 Transformer 是核心, 但工程价值由采样器和服务层兑现。

模型主干: encoder-like、decoder-like 与 hybrid

多数掩码扩散模型使用双向上下文, 因此它们的注意力更像 encoder。LLaDA 使用 vanilla Transformer 预测被 mask 的 token; LLaDA-V 在此基础上加入视觉编码器和 MLP connector, 把视觉特征投影到语言嵌入空间。[S05, S22] Dream 与 DiffuLLaMA 一类路线则强调从已有 AR 模型适配过来, 降低从零预训练的成本。[S11, S15]

hybrid 架构把 AR 与扩散放在同一张图里。Block Diffusion 在块之间保持自回归, 在块内使用离散扩散, 从而兼顾可变长度与块内并行。[S18] TiDAR 把「diffusion thinking」与「autoregressive talking」做成同一模型中的两种注意力模式, 目标是拿到 AR 质量与扩散吞吐之间的折中。[S25]

训练入口：从零训练与 AR 适配

路线	做法	好处	代价
从零预训练	直接用扩散目标训练大模型，如 LLaDA 8B、LLaDA-MoE	目标一致，架构和噪声 schedule 可统一设计	训练成本高，数据与稳定性要求接近大模型预训练
AR 权重适配	把 GPT2、LLaMA、Qwen 等 AR 权重持续训练成 diffusion LM	成本低，继承已有语言能力与生态	可能保留 AR 偏置，需要减弱左到右惯性
块级混合	块之间串行，块内扩散	支持 KV Cache、可变长度和并行采样	系统复杂度提高，块大小成为重要超参
服务型专用模型	针对代码、编辑、压缩、路由等场景训练	更容易在延迟和成本上形成产品优势	通用能力可能弱于同成本 AR frontier 模型

服务入口：API 兼容不是实现等价

Inception 的 Mercury 强调 OpenAI-compatible API 和企业落地，这对应用层很关键：调用方可以用熟悉的 chat completion 接口接入。[S13] 但服务内部不等价于 AR 模型。AR 模型天然流式吐 token；扩散模型通常先在一个块或序列上迭代，再释放若干 token。要提供流式体验，需要块级生成、渐进释放、预测长度估计和缓存管理共同配合。

05 · TRAINING STACK

| 训练技术栈

训练 diffusion LLM 的难点不是把 loss 写出来，而是让噪声分布、数据格式、模型容量、后训练目标和推理采样保持一致。训练时随机 mask，推理时置信度释放；二者不一致会带来分布偏移。

预训练数据与任务格式

预训练数据仍然需要大规模自然语言、代码、数学、指令和多语言语料。扩散目标改变的是「如何从样本中构造训练对」：同一条文本可以在不同 mask ratio 下形成多个训练样本。若做代码模型，数据还要包含 fill-in-the-middle、单元测试、补全上下文和编辑任务；若做多模态模型，视觉 token、图像 caption、视觉问答和文本到图像生成任务会混合进同一训练框架。

阶段	输入构造	目标	常见坑
Base 预训练	随机 mask 全文, 不区分 prompt 与 response	语言建模、知识、语法、代码模式	mask schedule 不合理会导致高噪声下学不到结构, 低噪声下泛化差
SFT	通常只 mask response, 让 prompt 作为条件	指令跟随、对话格式、工具调用模板	如果 prompt 也被过度 mask, 模型会在真实对话中丢失条件
代码专训	函数、文件、上下文前后缀、测试用例	补全、修复、重构、FIM	严格语法与跨位置依赖对采样器要求很高
多模态	图像 token 或视觉特征 + 文本 token	理解、生成、跨模态推理	模态对齐和扩散步数会同时影响文本与视觉输出

后训练: SFT、偏好优化与 RL

自回归模型中的 RLHF、DPO、GRPO 不能原封不动迁移到 diffusion LLM, 因为序列 log probability 的估计方式不同。LLaDA 1.5 提出 VRPO, 围绕 ELBO-based likelihood estimator 的高方差问题做偏好优化; d1 提出 diffu-GRPO, 用 masked SFT 与 critic-free policy gradient 提升数学和逻辑推理; Apple 的 DiffuCoder 使用 coupled-GRPO, 用互补 mask 噪声降低代码 RL 的方差。[S07, S24, S16]

后训练的核心问题: 奖励评估的是最终答案, 但 diffusion 训练过程有多轮中间状态。若只看最终 token, 很难分辨哪一轮、哪个 mask 决策、哪个 token 释放造成收益或错误。

训练基础设施

层级	建议组件	为什么重要
数据管道	语料去重、质量打分、代码许可过滤、任务标签、长度分桶	扩散训练会重复抽取不同噪声视图, 脏数据影响会被放大
训练框架	PyTorch、Transformers、自定义 Trainer、FSDP 或 Megatron 并行	模型主干可复用, 但 loss、collator、time embedding 需要改写
扩散组件	mask scheduler、noise sampler、loss weighting、x0 或 score 参数化	这些组件决定训练和推理是否一致
评测	lm-evaluation-harness、HumanEval、MBPP、LiveCodeBench、MMLU、GPQA、格式遵循	只看 perplexity 会低估编辑、填中间和并行生成价值
可复现框架	dLLM framework、LLaDA/Dream 官方仓库、CoDA 训练管线	2026 年的开源需求从「能跑 demo」升级到「可训练、可评测、可部署」[S26]

推理技术栈

diffusion LLM 的推理优化可以概括为一句话：用尽可能少的昂贵全模型前向，释放尽可能多的可靠 token。所有技术都围绕「少算、复用、先易后难、错误可回退」展开。

采样器的四个旋钮

旋钮	影响	工程判断
去噪步数 T	步数越多，质量通常越稳；步数越少，吞吐越高	代码、数学、结构化输出倾向更多步；闲聊和摘要可降低步数
每步释放比例	释放越多越快，但位置依赖可被破坏	用置信度阈值比固定比例更稳
remasking 策略	决定哪些 token 被保留，哪些回到 mask	entropy、margin、top1 confidence 是常见实用策略
温度与约束	影响多样性、格式遵循和错误率	扩散模型的 temperature 会同时改变 token 选择和生成顺序，不能按 AR 经验照搬[S16]

KV Cache: 扩散模型最现实的工程难点

AR 模型的 KV Cache 很自然：历史 token 不再变化，后续每一步只追加新 token。掩码扩散模型的未完成 token 会反复变化，双向 attention 让一个位置的变化影响全局，传统缓存难以直接复用。Fast-dLLM 的思路是引入 block-wise approximate KV Cache，并用 confidence-aware parallel decoding 降低多 token 同时解码带来的依赖破坏；论文在 LLaDA 与 Dream 上报告最高 27.6× 吞吐提升。[S17]

DPad 从另一个方向减少冗余计算：扩散模型每步预测未来 suffix 的大量 token，但最终只保留少部分。DPad 用 sliding window 和 distance-decay dropout 缩小关注的 suffix 范围，报告在 LLaDA-1.5 和 Dream 上可获得最高 61.4× speedup，同时保持可比精度。[S19] 这些数字不应被理解成所有场景的实际收益，而应理解成方向信号：推理优化已成为 diffusion LLM 是否能生产化的主战场。

FIGURE 3 SAMPLING LOOP

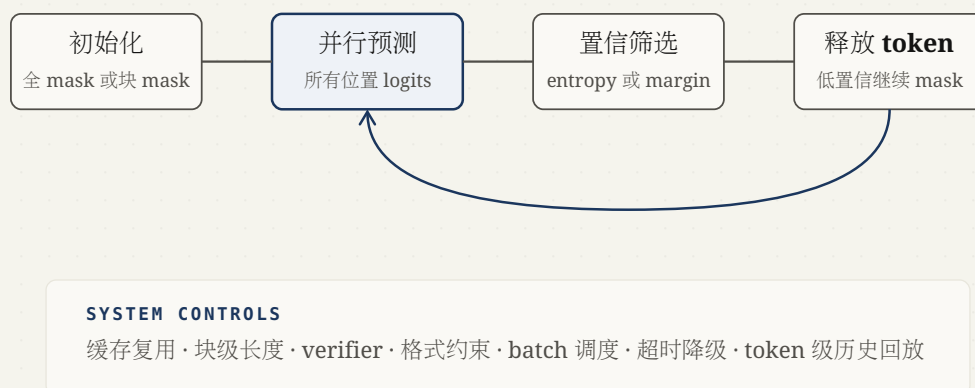


图 3：生产级推理不是单个采样公式，而是采样器、缓存、约束和调度的闭环。

延迟指标要分解

报告 diffusion LLM 性能时，应至少拆出四个指标：first visible token latency、平均采样速度、端到端延迟、质量保持率。Google DeepMind 的 Gemini Diffusion 页面明确区分 sampling speed excluding overhead 与 overhead，给出 1479 tokens/s 和 0.84 秒 overhead。[S14] 这比只写「每秒多少 token」更严谨，因为真实产品还要付 prompt 编码、调度、网络、后处理和 safety 检查的成本。

07 · MODEL LINEAGE

主流模型谱系

diffusion LLM 的发展不是单线突破，而是四条线同时推进：理论离散化、训练目标简化、模型规模化、工程产品化。理解谱系比记模型名更重要。

时间线

时间	代表工作	核心贡献	读者应记住的判断
2021	D3PM	结构化离散扩散，吸收态 [MASK]，连接 BERT 与 AR	语言扩散可以在离散状态空间中定义
2022	Diffusion-LM	连续嵌入扩散，可做细粒度可控生成	扩散的编辑和控制能力很早就显现
2023	SEDD	score entropy，把 score matching 推到离散空间	离散扩散的训练目标开始变强
2024	MDLM	简化掩码扩散目标，接近 AR perplexity	masked diffusion 成为主流大模型路线
2025	LLaDA、Dream、Block Diffusion、d1	8B 级模型、AR 适配、块级混合、RL reasoning	扩散语言模型进入大模型训练与后训练阶段
2025-2026	Mercury、Gemini Diffusion、CoDA、DiffuCoder、LLaDA-MoE/LLaDA2.0	商业 API、实验研究模型、代码模型、MoE 扩展	主战场转向生产延迟、代码、编辑、多模态与规模化

核心模型对照

模型/工作	机构或团队	路线	公开材料中的关键信息
LLaDA	RUC/GSAI、Ant 相关合作线索	从零训练的 masked diffusion	8B 规模；GitHub 称其 rivaling LLaMA3 8B；开源 Base 与 Instruct 权重[S05, S06]
LLaDA 1.5	LLaDA 团队	偏好优化	VRPO 降低 preference optimization 中 ELBO 估计方差；数学、代码、对齐 benchmark 有提升[S07]
LLaDA-MoE / LLaDA2.0	RUC 与 Ant Group / InclusionAI	MoE 扩展	LLaDA-MoE 7B 总参数、约 1.4B active；LLaDA2.0 仓库称 16B 与 100B MoE 权重开源[S08, S09, S10]
Dream 7B	HKU NLP、Huawei Noah's Ark Lab 相关合作	AR 权重初始化 + 离散扩散	7B 开源模型；Dream 仓库提供 inference、eval、SFT 代码；支持 entropy 等采样策略[S11]
Mercury	Inception Labs	商业 dLLM	Mercury Coder 宣称 H100 上超过 1000 tokens/s；General Chat 模型接入 playground/API/OpenRouter[S12, S13]
Gemini Diffusion	Google DeepMind	实验 text diffusion 模型	官方页面称其从随机噪声生成文本或代码；报告 1479 tokens/s sampling speed 和多项代码 benchmark[S14]
DiffuCoder	Apple ML Research 相关作者，HKU 合作	代码 dLLM + RL	7B code dLLM，130B code tokens，提出 coupled-GRPO，EvalPlus +4.4%[S16]
CoDA	Salesforce AI Research	1.7B coding diffusion	开源模型与训练管线；面向 code generation 与 bidirectional context understanding[S20]
MMaDA / LLaDA-V / LaViDa	多机构研究团队	多模态 diffusion LM	把 masked diffusion 主干扩展到视觉理解、文本到图像生成或统一多模态任务[S21-S23]

如何读 benchmark

diffusion LLM 的 benchmark 容易被误读。第一，代码 benchmark 与通用推理 benchmark 不同；Mercury Coder 在代码吞吐上强，不等于所有开放域问答都强。[S12] 第二，sampling speed 与端到端 latency 不同；Gemini Diffusion 页面把二者拆开，是更可取的口径。[S14] 第三，开源模型的默认采样参数可能没有调优；Dream 文档中 steps、alg、temperature 都会显著改变输出质量。[S11]

国内外厂商与开源生态

截至 2026-05-22，可公开核验的 diffusion LLM 版图呈现一个特点：大厂未全部公开押注，但已经出现明确的产品化、研究原型和开源权重三类路径。

国际厂商

机构	公开工作	路线判断
Google DeepMind	Gemini Diffusion experimental research model；官方展示 rapid response、iterative refinement 与 benchmark 表格	大厂级研究信号最明确，但当前定位仍是实验 demo 与 future models 的探索[S14]
Inception Labs	Mercury Coder、General Chat、Mercury 2、API 与企业部署	商业化最积极，产品叙事集中在低延迟、高吞吐、subagent、代码与企业流程[S12, S13]
Apple ML Research	Scaling Diffusion Language Models via Adaptation from AR Models；DiffuCoder	关注从已有 AR 模型适配和代码生成，研究材料严谨，部分工作说明「work done while at Apple」[S15, S16]
Salesforce AI Research	CoDA 1.7B code diffusion model，模型、评测和训练管线开源	走轻量代码助手路线，强调 bidirectional context 与低资源可复现[S20]
NVIDIA Research	TiDAR 等 hybrid 方向	更偏服务效率与架构融合：用 diffusion 做并行 drafting，用 AR 保持输出质量[S25]

国内与华人研究生态

机构/团队	公开工作	意义
中国人民大学 GSAI / Ant Group / InclusionAI	LLaDA、LLaDA-MoE、LLaDA2.0 系列	从 8B dense 到 MoE、100B 公开权重叙事，代表国内 diffusion LLM 规模化主线[S06, S08, S09, S10]
HKU NLP / Huawei Noah's Ark Lab	Dream 7B、Dream-Coder、DreamOn	AR 初始化 + diffusion 训练路线，强调规划、任意顺序、可调质量速度和开放代码[S11]
Tencent AI Lab	参与 Scaling Diffusion Language Models via Adaptation from AR Models	说明国内大厂研究团队也在探索从 AR 权重迁移到 DLM 的路径[S15]
ByteDance Seed 相关多模态生态	MMaDA 相关公开材料列出 ByteDance Seed 参与者	多模态 diffusion foundation model 方向值得关注，但应以论文作者与官方项目页为准[S21]
高校与开源社区	dLLM framework、Fast-dLLM、DPad、Block Diffusion 等	补足训练、评测、缓存、采样和部署工具，决定模型能否被工程团队真正采用[S17, S18, S19, S26]

对 OpenAI、Anthropic、Meta、Microsoft 等公司，公开材料中没有足够证据表明其已发布主线 diffusion LLM 产品。本文不把传闻或招聘猜测写成事实；如果未来出现官方模型卡、论文或 API 文档，应重新修订本章。

生态成熟度判断

生态成熟度可以按四级看：第一级是论文与小模型，证明目标函数可行；第二级是开源权重与 demo，证明模型能被社区复现；第三级是训练、评测、部署管线，证明工程团队可定制；第四级是商业 API、SLA、成本模型和真实客户案例。2026 年的 diffusion LLM 已跨过第二级，部分项目进入第三级和第四级，但离 AR LLM 的工具链成熟度仍有差距。

| 能力边界与失败模式

diffusion LLM 最容易被过度宣传成「更快、更会思考、还能修改」。这些说法都有真实来源，也都有边界。严谨判断要把速度、质量、可控性、稳定性和成本拆开。

五类失败模式

失败模式	表现	原因	缓解方向
依赖破坏	多个 token 并行释放后，局部看合理，全局不一致	条件独立假设削弱 token 间强依赖	confidence-aware decoding、verifier、块级生成
长度控制困难	需要预估输出长度，或块结束策略不稳定	经典扩散从固定长度噪声开始	Block Diffusion、半自回归扩展、EOS 策略
格式漂移	JSON、代码括号、工具调用参数不闭合	中间状态反复改写，约束未融入采样器	语法约束 decoding、repair loop、结构 verifier
缓存收益不足	理论并行但真实延迟不如预期	每轮全序列前向，KV Cache 难复用	近似 cache、DPA、块级 cache、模型调度
对齐方差高	偏好优化或 RL 不稳定，奖励提升难复现	序列概率估计和中间状态 credit assignment 更复杂	VRPO、diffu-GRPO、coupled-GRPO、过程奖励

哪些场景最先落地

最适合 diffusion LLM 的场景有三个共同点：需要快速生成较长块；允许模型先形成草稿再修正；任务可以从双向上下文或填中间中受益。代码补全、代码编辑、文档重写、批量摘要、上下文压缩、工具检索摘要、格式化转换、低延迟语音代理后端都符合这些条件。Inception 的 Mercury 2 案例把 context compaction 和 subagent 作为重点，正是因为这些任务对低延迟和结构化输出敏感，但不一定要求最强的 frontier reasoning。[S13]

哪些场景仍需谨慎

严格逐步推理、极长链条数学证明、必须 token-by-token 流式展示的交互、强依赖格式生成、高风险事实问答，都需要谨慎评估。Gemini Diffusion 在 Google 官方 benchmark 中，代码项表现接近 Flash-Lite，但 GPQA Diamond、BIG-Bench Extra Hard、Global MMLU Lite 等项目低于 Gemini 2.0 Flash-Lite。[S14] 这说明 diffusion 架构的速度与编辑优势不自动转化为所有认知 benchmark 的优势。

EVALUATION RULE

评测 diffusion LLM 时应同时跑三类任务：生成质量任务、编辑/填中间任务、延迟/成本任务。只跑传统 QA benchmark 会漏掉它的长处；只跑 tokens/s 会漏掉它的短处。

10 · ROADMAP

未来路线图

2026-2028 年 diffusion LLM 的发展不会是「取代自回归」这么简单。更可能出现混合架构、专用模型和服务层路由：AR 负责高精度逐步推理，diffusion 负责低延迟块生成、编辑、压缩和多模态统一。

路线一：更强的采样与缓存

最直接的收益来自推理层。Fast-dLLM、DPad、Block Diffusion 等工作都在解决同一个问题：怎样减少无效前向，怎样在 bidirectional 模型里复用上下文，怎样在不破坏依赖的前提下一次释放更多 token。[S17-S19] 未来会出现更标准化的 diffusion serving engine，类似今天 vLLM、TensorRT-LLM、SGLang 对 AR 模型的作用。

路线二：AR 适配和 hybrid 成为低成本入口

从零训练 8B、100B diffusion 模型成本很高。Apple 相关工作展示了把 GPT2 与 LLaMA 适配为 DiffuGPT、DiffuLLaMA 的路线，并称 127M 到 7B 级模型可用少于 200B tokens 训练成 DLM。[S15] 对多数团队来说，从强 AR base 迁移、做特定任务后训练、配合高质量采样器，比从零预训练更现实。

路线三：代码、编辑与 Agent 子任务先商业化

代码任务天然适合 diffusion：未来上下文和过去上下文都重要，fill-in-the-middle 是刚需，unit test 可做 verifier。Mercury Coder、DiffuCoder、CoDA 都押注代码方向。[S12, S16, S20] Agent 系统中的 compaction、routing、tool summary、handoff 也适合用高速模型承担，Mercury 2 的公开案例已经把方向产品化。[S13]

路线四：多模态统一会推动 diffusion 主干

图像、视频和音频领域已经熟悉扩散生成。若语言侧也能稳定使用扩散，统一多模态生成的架构会更自然。MMaDA、LLaDA-V、LaViDa、Dream-VL/Dream-VLA 等工作说明研究界正在把 masked diffusion backbone 扩展到视觉理解、文本到图像和机器人行动。[S21-S23] 难点在于不同模态的噪声 schedule、token 粒度和奖励函数不一致。

路线五：评价体系会重写

AR 模型的评价习惯偏向 next-token likelihood、标准 QA、单次生成质量。diffusion LLM 需要评价更多维度：任意顺序生成是否真的利用了后文；编辑任务能否比 AR 模型少重写；同等质量下端到端延迟是否更低；格式错误能否在中间步被修正；采样步数降低后质量曲线如何退化。没有这些指标，就无法判断 diffusion 的架构红利是否真实。

前景判断：diffusion LLM 短期最可能成为「高速可编辑生成层」和「Agent 系统中的子模型层」；中期会与 AR 模型形成 hybrid；长期能否成为通用主干，取决于可变长度、缓存、对齐、推理能力和工具链成熟度。

11 · LEARNING PATH

学习路径、术语表与参考资料

学 diffusion LLM 最怕直接跳进最新模型名。正确路径是先理解离散扩散，再理解 masked diffusion 目标，然后跑通开源模型，最后研究采样与服务优化。

四周入门路线

阶段	学习目标	建议材料	产出
第 1 周	建立扩散、离散状态、[MASK] 吸收态直觉	D3PM、Diffusion-LM、SEDD 摘要和图示	手写一个 toy masked diffusion sampler
第 2 周	理解 MDLM 与 LLaDA 的训练目标	MDLM、LLaDA、LLaDA GitHub	用小语料训练 100M 以下模型或复现推理
第 3 周	掌握采样器和推理参数	Dream、Fast-dLLM、DPad、Block Diffusion	比较 random、entropy、margin 三类 remasking
第 4 周	理解后训练和生产化	LLaDA 1.5、d1、DiffuCoder、CoDA、Mercury、Gemini Diffusion	写一个应用评测表：质量、延迟、成本、格式错误率

术语表

diffusion LLM / DLM：把语言生成建模为多步去噪过程的语言模型，常见实现为 masked diffusion。

masked diffusion：前向过程把 token 逐步变成 [MASK]，反向过程学习逐步恢复 token。

remasking：采样中把低置信 token 重新置为 [MASK]，让模型后续步骤继续修正。

noise schedule：控制不同时间步噪声强度或 mask 比例的函数。

x0 prediction：模型直接预测干净样本 x_0 的参数化方式。

score entropy: SEDD 提出的离散 score matching 目标, 用于估计数据分布比值。

KV Cache: Transformer 推理中复用历史 key/value 的缓存; 在 bidirectional diffusion 中不如 AR 自然。

Block Diffusion: 块之间自回归、块内扩散的混合范式, 用于改善可变长度与缓存复用。

VRPO / diffu-GRPO / coupled-GRPO: 面向 diffusion LLM 的偏好优化或强化学习变体, 核心目标是降低概率估计方差并适配多步去噪。

核心参考资料

ID	资料	用途
S01	Austin et al., Structured Denoising Diffusion Models in Discrete State-Spaces, 2021	离散扩散与 D3PM 基础
S02	Li et al., Diffusion-LM Improves Controllable Text Generation, 2022	连续语言扩散与可控生成
S03	Lou et al., Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution, 2023	SEDD 与 score entropy
S04	Sahoo et al., Simple and Effective Masked Diffusion Language Models, 2024	MDLM 与简化训练目标
S05	Nie et al., Large Language Diffusion Models, 2025	LLaDA 主论文
S06	ML-GSAI/LLaDA GitHub repository	LLaDA 权重、推理与评测代码
S07	Zhu et al., LLaDA 1.5, 2025	VRPO 与偏好优化
S08	Zhu et al., LLaDA-MoE, 2025	MoE diffusion LM
S09	RUC and Ant Group LLaDA-MoE announcement	国内机构合作与 MoE 扩展
S10	inclusionAI/LLaDA2.X	Ant Group LLaDA2.0 开源线索
S11	DreamLM/Dream GitHub and Dream 7B paper	Dream 7B、Dream-Coder、采样参数
S12	Inception Labs Mercury Coder	商业代码 dLLM 与速度指标
S13	Inception Labs Mercury general chat and Mercury 2	API、企业应用与 subagent 场景
S14	Google DeepMind Gemini Diffusion page	大厂实验 research model 与速度指标
S15	Apple ML Research, Scaling Diffusion Language Models	从 AR 权重适配到 DLM
S16	Apple ML Research, DiffuCoder	代码 dLLM 与 coupled-GRPO
S17	Fast-dLLM	KV Cache 与并行解码优化
S18	Block Diffusion	块级混合、可变长度与缓存复用
S19	DPad	Suffix dropout 与长序列推理加速
S20	Salesforce CoDA model card and arXiv paper	轻量代码 diffusion model
S21	MMaDA	统一多模态 diffusion foundation model
S22	LLaDA-V	视觉指令调优的 diffusion MLLM
S23	LaViDa	多模态理解中的 diffusion language backbone
S24	d1	diffu-GRPO 与 reasoning 后训练
S25	TiDAR	diffusion 与 AR 混合架构
S26	dLLM: Simple Diffusion Language Modeling, 2026	标准化训练、推理、部署框架

最后的判断框架

1. 若任务是严格左到右推理，并且输出不长，AR 模型仍是默认选择。
2. 若任务需要大量编辑、填中间、低延迟长块输出、上下文压缩或代码补全，应评估 diffusion LLM。
3. 若团队没有能力改采样器和服务层，只下载权重通常不够；至少要掌握 remasking、步数、缓存和评测。
4. 若供应商只给 tokens/s，不给端到端延迟、质量保持率和硬件条件，应视为不完整 benchmark。

持续复核建议

建议每季度复核三组变量：第一，Google、Inception、Apple、Ant Group、Dream 团队等路线是否从研究展示走向稳定 API、开源权重或生产案例；第二，Fast-dLLM、DPad、Block Diffusion、dLLM framework 等推理工具是否形成可复用 serving stack；第三，代码、编辑、多模态和 Agent 子任务是否在端到端成本上持续优于同级 AR 模型。只有这三件事同时改善，diffusion LLM 才会从「有趣的新范式」变成「值得纳入基础设施规划的工程选项」。